



GUIA DEL ESTUDIANTE

FECHA:	Del 20 al 26 de octubre de 2025
UNIDAD A ESTUDIAR	<i>Programación Orientada a Objetos en Desarrollo Moderno.</i>
DURACIÓN DE LA UNIDAD	Tres semanas.
OBJETIVOS	Al final de esta sección, el estudiante podrá: Conceptos Fundamentales de Arquitectura • Explicar los principios de la Arquitectura Limpia y por qué es importante separar las responsabilidades en tu código. • Diseñar aplicaciones usando la arquitectura en capas (Controlador, Servicio, Repositorio), sabiendo qué lógica va en cada capa. • Diferenciar entre una Entidad (la tabla de la base de datos) y un DTO (los datos que viajan por la API), y entender por qué se usan ambos.

Programación Moderna y Bases de Datos

- Utilizar expresiones Lambda y Streams de Java para manejar listas y colecciones de datos de una manera más limpia y eficiente.
- Implementar la capa de acceso a datos con Spring Data JPA y Spring Data MongoDB, creando repositorios que te ahorran escribir código repetitivo.
- Crear consultas a la base de datos de forma automática, simplemente nombrando los métodos de tu repositorio de una manera específica.

Habilidades Prácticas y Conectividad

- Configurar tu aplicación Spring Boot para que se conecte a diferentes bases de datos como H2 (para pruebas rápidas), MySQL (para producción) y MongoDB (para datos NoSQL).
- Construir una API REST funcional con endpoints **GET** (para obtener datos) y **POST** (para crear datos) que siga una arquitectura profesional.
- Aplicar correctamente las anotaciones más importantes como **@Entity**, **@Id**, **@Document** y **@Repository** para que tus clases de Java se puedan guardar en una base de datos.

IMPORTANTE

Verificar las orientaciones antes de cada sesión sincrónica y asegúrese de preguntar cualquier duda que tengas sobre las diferentes actividades que se asignarán.

CONTENIDO DE LA SEMANA:

Unidad:	<i>3 - Programación Orientada a Objetos en Desarrollo Moderno</i>
Semana:	<i>Once</i>
Tema a desarrollar	<i>3.11 Principios de arquitectura limpia en Spring Boot.</i> <i>3.12 Separación en capas: controlador, servicio, repositorio.</i> <i>3.13 Streams y Lambdas</i> <i>3.14 Uso de Spring Data JPA.</i> <i>3.15 Conexión con base de datos H2 (en memoria).</i> <i>3.16 Anotaciones: @Entity, @Id, @GeneratedValue, @Repository.</i>
Material Base:	<i>Presentación de PowerPoint, donde se encuentra información relevante de los temas de la semana once.</i> <i>Material de lectura , el cual proporciona información más compleja de los temas a estudiar.</i>
Material de apoyo	N/A
Actividades y Tareas (si aplica):	N/A
Fechas límite de entrega (si aplica):	N/A

Recursos Adicionales:	N/A
Vocabulario útil (si aplica).	Arquitectura y Diseño Arquitectura Limpia (Clean Architecture): Un enfoque de diseño que separa el software en capas para

	que la lógica de negocio principal sea independiente de los frameworks y la base de datos. • Separación de Responsabilidades (Separation of Concerns): Principio que dicta que cada parte de la aplicación debe tener una única y bien definida responsabilidad. • Capa (Layer): Un nivel lógico de la aplicación. En nuestros ejemplos: Capa de Presentación (Controlador), Capa de Negocio (Servicio) y Capa de Datos (Repositorio). • Lógica de Negocio (Business Logic): Las reglas y procesos que son el corazón de la aplicación. Por ejemplo, "un producto no puede tener un precio negativo". • DTO (Data Transfer Object): Un objeto simple cuya única función es transportar datos entre capas, especialmente entre el cliente y el controlador. Ayuda a no exponer la estructura interna de la base de datos. • Entidad (Entity): Una clase Java que está mapeada directamente a una tabla en una base de datos relacional (anotada con <code>@Entity</code>). • Desacoplamiento (Decoupling): El objetivo de la buena arquitectura. Significa que las capas son independientes entre sí, por lo que un cambio en una (como cambiar de MySQL a MongoDB) no obliga a cambiar las otras. Persistencia y Bases de Datos • JPA (Jakarta Persistence API): El estándar o especificación en Java para gestionar datos en bases de datos relacionales. Es como un "contrato" que herramientas como Hibernate implementan. • ORM (Object-Relational Mapping): La técnica que "traduce" objetos de Java a filas en una tabla de base de datos relacional, y viceversa. Es lo que JPA nos permite hacer. • Repositorio (Repository): Una interfaz que define las operaciones de acceso a datos (CRUD). Spring Data JPA crea la implementación por nosotros.
--	--

- Consulta Derivada (Derived Query): La "magia" de Spring Data que crea automáticamente una consulta SQL a partir del nombre de un método en el repositorio (ej. `findBySku`).
- JDBC (Java Database Connectivity): La tecnología base que permite a las aplicaciones Java conectarse y ejecutar consultas en bases de datos SQL.
- Documento (Document): El equivalente a una **Entidad** en el mundo NoSQL (como MongoDB). Es un registro de datos, usualmente en un formato similar a JSON.

Spring Framework

- Bean: Un objeto que es creado y gestionado por el contenedor de Spring. En nuestro caso, los controladores, servicios y repositorios son **beans**.
- Anotación (Annotation): Metadatos (ej. `@Service`, `@RestController`, `@Autowired`) que se añaden al código para darle instrucciones a Spring sobre cómo debe comportarse.
- Inyección de Dependencias (Dependency Injection): El mecanismo por el cual Spring "inyecta" automáticamente una instancia de una clase (como un **ProductoRepository**) dentro de otra que la necesita (como un **ProductoService**).

Programación y API

- Lambda (Expresión Lambda): Una forma corta y anónima de escribir una función. Es la base de la programación funcional en Java. Se reconoce por la flecha `->`.
- Stream API: Una herramienta de Java 8+ para procesar colecciones de datos (como listas) de una manera declarativa y fluida, usando operaciones como `filter()` y `map()`.
- API RESTful: Un estilo de arquitectura para construir servicios web. Se basa en el protocolo HTTP y utiliza sus verbos (`GET`, `POST`, `PUT`, `DELETE`) para operar sobre recursos.

	• Endpoint: Una URL específica de nuestra API donde se puede realizar una operación (ej. POST /api/productos). • JSON (JavaScript Object Notation): El formato de texto estándar para el intercambio de datos en las APIs modernas. Es ligero y fácil de leer tanto para humanos como para máquinas.
--	---

RECORDATORIOS

- ✓ Escriba todas las dudas en su cuaderno o libreta de apuntes y pregúntele a su docente tutor mientras se desarrolla la sesión sincrónica o escríbale a su correo institucional.
- ✓ Esté atento en sus sesiones sincrónicas porque su docente tutor puede pedirle ejemplos de las temáticas que se estén desarrollando.
- ✓ Si tiene preguntas o dudas durante el desarrollo de esta unidad, contacte a su docente tutor y solicitar asistencia para aclarar las dudas que tenga.